



امنیت داده و شبکه

نیم‌سال دوم ۱۴۰۴-۰۵

دکتر مهدی خرازی و دکتر مرتضی امینی

پاسخ‌دهنده: معین آعلی - ۴۰۱۱۰۵۵۶۱

تمرین سوم

فهرست مسائل

۱		مسئله ۱
۳		مسئله ۲
۳		رمزنگاری RSA
۴		تبادل کلید Diffie-Hellman
۶		مسئله ۳
۶		احراز هویت پیام‌های با طول مشخص توسط فرستنده
۶		محدودیت طول در سمت گیرنده
۷		مسئله ۴
۷		نسخه اول
۷		نسخه دوم
۸		نسخه سوم
۸		راه حل نهایی
۹		مسئله ۵
۹		روش‌های ترکیب MAC و رمزنگاری
۹		محدوده اعمال MAC روی بسته‌های شبکه
۱۰		مسئله ۶
۱۰		استخراج فرمول‌های بازیابی
۱۰		محاسبه کلید خصوصی
۱۱		مکانیزم استاندارد جلوگیری از استفاده مجدد Nonce
۱۲		مسئله ۷
۱۲		تحلیل دامنه نفوذ
۱۲		ابطال گواهی‌های جعل شده
۱۳		بهبودهای پیشنهادی معماری PKI

پاسخ مسئله‌ی ۱.

رابطه رمزگشایی برای بلوک i -ام به صورت زیر تعریف می‌شود:

$$P_i = D_K(C_i) \oplus C_{i-1}$$

که در آن $IV = C_0$ است. با توجه به این رابطه، خراب شدن یک بلوک از متن رمز (C_i) تنها بر روی رمزگشایی دو بلوک از متن آشکار اثر می‌گذارد:

۱. بلوک هم‌شمار (P_i) : به دلیل حضور C_i به عنوان ورودی مستقیم تابع رمزگشایی D_K .

۲. بلوک بعدی (P_{i+1}) : به دلیل XOR شدن C_i با خروجی تابع رمزگشایی بلوک بعدی.

خطا به بلوک‌های P_{i+2} و بعد از آن منتشر نمی‌شود، زیرا C_{i+1} سالم است.

حال با فرض خراب شدن ۴ بلوک از متن رمز، دو سناریو برای حداقل و حداکثر خرابی در متن آشکار وجود دارد:

• حداکثر تعداد بلوک‌های خراب:

اگر ۴ بلوک خراب در متن رمز کاملاً غیرمتوالی و پراکنده باشند (به طوری که خطاهای آنها با هم تداخل نداشته باشد)، هر بلوک خراب متن رمز، دقیقاً ۲ بلوک از متن آشکار را خراب می‌کند. در نتیجه:

$$\text{حداکثر تعداد بلوک‌های خراب} = 4 \times 2 = 8$$

• حداقل تعداد بلوک‌های خراب:

اگر ۴ بلوک خراب به صورت متوالی و پشت‌سرهم ظاهر شوند (مثلاً $C_i, C_{i+1}, C_{i+2}, C_{i+3}$)، اثر خطای بلوک‌های قبلی روی اثر تخریبی تابع رمزگشایی بلوک‌های بعدی هم‌پوشانی پیدا می‌کند. در این حالت، بلوک‌های P_i تا P_{i+3} به خاطر ورودی نامعتبر تابع دکریپت، و بلوک P_{i+4} به خاطر XOR با C_{i+3} خراب می‌شوند. به طور کلی برای n بلوک متوالی خراب، $n + 1$ بلوک آشکار خراب می‌شود. در نتیجه:

$$\text{حداقل تعداد بلوک‌های خراب} = 4 + 1 = 5$$

پاسخ مسئله‌ی ۲.

رمزنگاری RSA

(آ) محاسبه کلید عمومی (n, e) و کلید خصوصی (n, d) :

ابتدا مقدار n را محاسبه می‌کنیم:

$$n = p \times q = 7 \times 13 = 91$$

سپس مقدار $\phi(n)$ را به دست می‌آوریم:

$$\phi(n) = (p-1)(q-1) = (7-1)(13-1) = 6 \times 12 = 72$$

برای یافتن کلید خصوصی d ، باید معکوس ضربی $e = 7$ را به پیمانه $\phi(n) = 72$ با استفاده از الگوریتم اقلیدسی تعمیم‌یافته پیدا کنیم. یعنی حل معادله هم‌نهشتی:

$$7d \equiv 1 \pmod{72}$$

مراحل الگوریتم تقسیم اقلیدسی (رو به جلو):

$$72 = 10 \times 7 + 2$$

$$7 = 3 \times 2 + 1$$

حالا مراحل را به صورت بازگشتی (رو به عقب) برای یافتن ترکیب خطی بنویسیم:

$$1 = 7 - 3 \times 2$$

جایگذاری $2 = 72 - 10 \times 7$:

$$1 = 7 - 3 \times (72 - 10 \times 7)$$

$$1 = 7 - 3 \times 72 + 30 \times 7$$

$$1 = 31 \times 7 - 3 \times 72$$

بنابراین:

$$31 \times 7 \equiv 1 \pmod{72} \implies d = 31$$

کلید عمومی: $(n, e) = (91, 7)$

کلید خصوصی: $(n, d) = (91, 31)$

(ب) رمزگذاری پیام $M = 82$ و محاسبه متن رمز نشده C :

فرمول رمزگذاری:

$$C \equiv M^e \pmod{n} \implies C \equiv 82^7 \pmod{91}$$

ابتدا پایه را ساده می‌کنیم: $82 \equiv -9 \pmod{91}$. پس:

$$C \equiv (-9)^7 \pmod{91}$$

$$(-9)^2 = 81 \equiv -10 \pmod{91}$$

$$(-9)^4 \equiv (-10)^2 = 100 \equiv 9 \pmod{91}$$

$$(-9)^6 = (-9)^4 \times (-9)^2 \equiv 9 \times (-10) = -90 \equiv 1 \pmod{91}$$

در نهایت:

$$C \equiv (-9)^7 = (-9)^6 \times (-9)^1 \equiv 1 \times (-9) = -9 \equiv 82 \pmod{91}$$

پس متن رمزگذاری شده برابر است با: $C = 82$

ج) رمزگشایی متن C و بازگشت به پیام اصلی M :

فرمول رمزگشایی:

$$M = C^d \pmod{n} \implies M = 82^{31} \pmod{91}$$

با توجه به این که $82 \equiv -9 \pmod{91}$ ، داریم:

$$M \equiv (-9)^{31} \pmod{91}$$

با استفاده از محاسبات بخش قبل می‌دانیم که $(-9)^6 \equiv 1 \pmod{91}$. بنابراین توان را به پیمانه ۶ ساده می‌کنیم:

$$31 = 5 \times 6 + 1$$

$$(-9)^{31} = ((-9)^6)^5 \times (-9)^1 \equiv (1)^5 \times (-9) = -9 \equiv 82 \pmod{91}$$

پس پیام رمزگشایی شده برابر با $M = 82$ است که با پیام اولیه کاملاً مطابقت دارد.

تبادل کلید Diffie-Hellman

آ) یافتن کلید خصوصی آلیس (X_A) در صورت داشتن کلید عمومی $Y_A = 9$:

فرمول کلید عمومی در پروتکل دیفی-هلمن عبارت است از:

$$Y_A = \alpha^{X_A} \pmod{q} \implies 9 = 2^{X_A} \pmod{11}$$

با آزمون مقادیر مختلف برای X_A (از ۱ تا ۱۰):

$$2^1 = 2 \pmod{11} \bullet$$

$$2^2 = 4 \pmod{11} \bullet$$

$$2^3 = 8 \pmod{11} \bullet$$

$$2^4 = 16 \equiv 5 \pmod{11} \bullet$$

$$2^5 = 32 \equiv 10 \pmod{11} \bullet$$

$$2^6 = 64 \equiv 9 \pmod{11} \bullet$$

بنابراین، کلید خصوصی آلیس برابر است با: $X_A = 6$

مسئله پیدا کردن X_A از روی Y_A ، α و q به نام مسئله لگاریتم گسسته شناخته می‌شود. برای اعداد اول بسیار بزرگ (q با چند صد رقم)، هیچ الگوریتم کلاسیک با زمان چندجمله‌ای برای حل این مسئله در زمان معقول وجود ندارد و امنیت این پروتکل نیز دقیقاً بر پایه همین دشواری استوار است.

ب) محاسبه کلید مشترک K_{AB} در صورت داشتن کلید عمومی باب $Y_B = 3$:

هر دو طرف می‌توانند به طور مستقل کلید مشترک را محاسبه کنند:

۱. محاسبه توسط آلیس (با استفاده از کلید عمومی باب Y_B و کلید خصوصی خود $X_A = 6$):

$$K_{AB} = Y_B^{X_A} \pmod{q} = 3^6 \pmod{11}$$

$$3^2 = 9 \equiv -2 \pmod{11}$$

$$3^4 \equiv (-2)^2 = 4 \pmod{11}$$

$$3^6 = 3^4 \times 3^2 \equiv 4 \times 9 = 36 \equiv 3 \pmod{11}$$

بنابراین کلید مشترک از نظر آلیس ۳ است.

۲. محاسبه توسط باب (ابتدا نیاز به یافتن کلید خصوصی خود X_B دارد):

$$Y_B = \alpha^{X_B} \pmod{q} \implies 3 = 2^{X_B} \pmod{11}$$

با ادامه بررسی توان‌های ۲ از بخش قبل:

$$2^7 = 2^6 \times 2 \equiv 9 \times 2 = 18 \equiv 7 \pmod{11} \bullet$$

$$2^8 = 2^7 \times 2 \equiv 7 \times 2 = 14 \equiv 3 \pmod{11} \bullet$$

پس کلید خصوصی باب $X_B = 8$ است. حالا باب کلید مشترک را با کلید عمومی آلیس ($Y_A = 9$) محاسبه می‌کند:

$$K_{AB} = Y_A^{X_B} \pmod{q} = 9^8 \pmod{11}$$

$$\text{از آنجا که } 9 \equiv -2 \pmod{11}:$$

$$9^8 \equiv (-2)^8 = 256 \pmod{11}$$

$$\text{با تقسیم } 256 \text{ بر } 11 \text{ داریم: } 256 = 23 \times 11 + 3 \text{ پس } 256 \equiv 3 \pmod{11}.$$

مشاهده می‌شود که هر دو طرف به طور مستقل به کلید مشترک یکسان $K_{AB} = 3$ رسیدند.

پاسخ مسئله‌ی ۳.

احراز هویت پیام‌های با طول مشخص توسط فرستنده

فرض کنید فرستنده فقط برای پیام‌های دو بلوکی تگ تولید می‌کند و گیرنده طول پیام را بررسی نمی‌کند. مهاجم برای جعل پیام چهاربلوکی $M = (m_1, m_2, m_3, m_4)$ به صورت زیر عمل می‌کند:

۱. پیام $M_1 = (m_1, m_2)$ را به فرستنده می‌دهد و تگ $t_1 = E_k(m_2 \oplus E_k(m_1))$ را دریافت می‌کند.

۲. سپس پیام $M_2 = (m_3 \oplus t_1, m_4)$ را به فرستنده می‌دهد و تگ $t_2 = E_k(m_4 \oplus E_k(m_3 \oplus t_1))$ را دریافت می‌کند.

۳. مهاجم ادعا می‌کند که t_2 تگ معتبر پیام $M = (m_1, m_2, m_3, m_4)$ است.

گیرنده برای محاسبه CBC-MAC پیام M خواهد داشت:

$$c_1 = E_k(m_1)$$

$$c_2 = E_k(m_2 \oplus c_1) = E_k(m_2 \oplus E_k(m_1)) = t_1$$

$$c_3 = E_k(m_3 \oplus c_2) = E_k(m_3 \oplus t_1)$$

$$c_4 = E_k(m_4 \oplus c_3) = E_k(m_4 \oplus E_k(m_3 \oplus t_1)) = t_2$$

بنابراین $CBCMAC(M) = t_2$ و گیرنده تگ را معتبر می‌پذیرد. پس CBC-MAC در صورت عدم کنترل طول پیام در برابر این جعل آسیب‌پذیر است.

محدودیت طول در سمت گیرنده

فرض کنید گیرنده فقط پیام‌های سه‌بلوکی را می‌پذیرد، اما فرستنده برای هر طولی تگ تولید می‌کند.

۱. مهاجم پیام $M_1 = (m_1, m_2)$ را ارسال کرده و تگ $t_1 = E_k(m_2 \oplus E_k(m_1))$ را دریافت می‌کند.

۲. سپس پیام تک‌بلوکی $M_2 = (m_3)$ را ارسال کرده و تگ $t_2 = E_k(m_3)$ را دریافت می‌کند.

۳. اکنون پیام $M^* = (m_1, m_2, m_3 \oplus t_1)$ را همراه با تگ t_2 برای گیرنده می‌فرستد.

گیرنده محاسبه می‌کند:

$$c_1 = E_k(m_1),$$

$$c_2 = E_k(m_2 \oplus c_1) = t_1,$$

$$c_3 = E_k((m_3 \oplus t_1) \oplus c_2) = E_k((m_3 \oplus t_1) \oplus t_1) = E_k(m_3) = t_2.$$

در نتیجه $CBCMAC(M^*) = t_2$ و گیرنده تگ را معتبر می‌پذیرد. بنابراین صرف محدود کردن طول پیام در سمت گیرنده نیز امنیت CBC-MAC را تضمین نمی‌کند.

پاسخ مسئله‌ی ۴.

نسخه اول

آسیب‌پذیری این نسخه ناشی از عدم وجود جداکننده (Delimiter) مناسب بین مقادیر الحاق شده است. فرمول ساخت MAC به صورت زیر است:

$$MAC = H(K \parallel \text{player_id} \parallel \text{gold_amount})$$

با توجه به فرض مسئله، حساب کاربری با نام eve1 درخواستی برای دریافت ۰ سکه فرستاده و MAC معتبر آن را دریافت کرده است. بنابراین مقداری که هش می‌شود برابر است با:

$$K \parallel \text{"eve1"} \parallel \text{"۰"} = K \parallel \text{"eve10"}$$

حال اگر هکر بخواهد برای اکانت اصلی خود یعنی eve مقدار ۱۰ سکه دریافت کند، مقدار ورودی هش سرور به صورت زیر خواهد بود:

$$K \parallel \text{"eve"} \parallel \text{"۱۰"} = K \parallel \text{"eve10"}$$

همان‌طور که مشاهده می‌شود، هر دو رشته پس از الحاق کاملاً یکسان تجمیع می‌شوند ($K \parallel \text{eve10}$). در نتیجه، MAC تولید شده برای درخواست اول، برای درخواست دوم نیز کاملاً معتبر است.

نحوه جعل: هکر کافی است دقیقاً همان MAC دریافت شده برای (eve1، ۰ سکه) را بردارد و همراه با درخواست جعلی خود یعنی (eve، ۱۰ سکه) به سرور ارسال کند. سرور به دلیل یکسان بودن حاصل الحاق، MAC را معتبر تشخیص می‌دهد.

نسخه دوم

در این نسخه فرمول به صورت زیر تغییر یافته است:

$$MAC = H(K) \oplus H(\text{player_id} \parallel \text{":"} \parallel \text{gold_amount})$$

مشکل اصلی این فرمول استفاده از عملگر XOR است. این عملگر خاصیت جابه‌جایی و بازگشتی دارد که به هکر اجازه می‌دهد مقدار ثابت $H(K)$ را کاملاً حذف یا ایزوله کند.

ما درخواست معتبر برای کاربری eve و مقدار ۵ سکه را شنود کرده‌ایم و MAC_{old} را در دست داریم:

$$MAC_{old} = H(K) \oplus H(\text{"۵:eve"})$$

از آنجا که مقدار $H(\text{"۵:eve"})$ بدون نیاز به کلید مخفی و صرفاً با داشتن تابع هش قابل محاسبه است، هکر می‌تواند با اعمال XOR این مقدار را از MAC_{old} حذف کرده و مقدار مستقل $H(K)$ را استخراج کند:

$$H(K) = MAC_{old} \oplus H(\text{"۵:eve"})$$

حال هکر می‌خواهد درخواست جدیدی برای دریافت ۹۹۹۹ سکه جعل کند. فرمول MAC جدید برابر است با:

$$MAC_{new} = H(K) \oplus H(\text{"۹۹۹۹:eve"})$$

با جایگذاری مقدار استخراج‌شده $H(K)$ ، MAC جعلی جدید به سادگی به دست می‌آید:

$$MAC_{new} = (MAC_{old} \oplus H(\text{"۵:eve"})) \oplus H(\text{"۹۹۹۹:eve"})$$

نسخه سوم

کد پیاده‌سازی شده در سرور دچار آسیب‌پذیری (Timing Attack) است. تابع با استفاده از یک حلقه for، کاراکترهای دو MAC را تک‌به‌تک از چپ به راست مقایسه می‌کند و به محض دیدن اولین عدم تطابق، مقدار False را برمی‌گرداند.

روند استخراج MAC توسط هکر:

۱. حدس کاراکتر اول: هکر MACهایی ارسال می‌کند که کاراکتر اول آن‌ها متفاوت است. اگر کاراکتر اول درست باشد، حلقه یک بار بیشتر اجرا شده و زمان پاسخ‌دهی سرور به اندازه یک پالس کوچک (اما قابل اندازه‌گیری) طولانی‌تر می‌شود. هکر با تحلیل آماری زمان پاسخ‌ها، کاراکتر اول MAC صحیح را پیدا می‌کند.
۲. حدس کاراکترهای بعدی: پس از تثبیت کاراکتر اول، هکر همین روند را برای کاراکتر دوم، سوم و... تکرار می‌کند تا زمانی که زمان پاسخ‌دهی به حداکثر خود برسد (یعنی کل MAC شده و حلقه تا انتها اجرا شود).

راه حل نهایی

۱. ساختار استاندارد جایگزین: به جای روش‌های ابداعی، توسعه‌دهندگان باید از ساختار استاندارد HMAC (Hash-based Message Authentication Code) استفاده کنند.
- این ساختار برای جلوگیری از حملاتی مثل Length Extension و آسیب‌پذیری‌های الحاق خام، کلید مخفی را در دو مرحله مجزا (با استفاده از دو پد داخلی و خارجی ipad و opad) با پیام ترکیب کرده و هش می‌کند:

$$HMAC(K, m) = H\left((K \oplus opad) \parallel H((K \oplus ipad) \parallel m)\right)$$

۲. رفع مشکل امنیتی نسخه سوم: برای جلوگیری از حمله زمان‌بندی، مقایسه دو رشته باید در زمان ثابت انجام شود؛ یعنی فارغ از اینکه خطا در کدام کاراکتر رخ داده است، کل رشته همیشه تا انتها بررسی شود.

پاسخ مسئله‌ی ۵.

روش‌های ترکیب MAC و رمزنگاری

۱. شرح دو روش بر اساس عملیات روی Plaintext و Ciphertext:

- روش اول MAC-then-Encrypt - MtE: در این روش ابتدا تگ MAC روی متن اصلی (P) محاسبه می‌شود. سپس کل مجموع شامل متن اصلی و تگ MAC با هم رمزنگاری می‌شوند تا متن رمز شده نهایی (C) تولید شود:

$$C = E_{k_1}(P \parallel \text{MAC}_{k_2}(P))$$

- روش دوم Encrypt-then-MAC - EtM: در این روش ابتدا متن اصلی (P) رمزنگاری می‌شود تا متن رمز شده (C) به دست آید. سپس تگ MAC روی این متن رمز شده محاسبه و به آن الحاق می‌گردد:

$$C = E_{k_1}(P), \quad t = \text{MAC}_{k_2}(C) \implies \text{Result} = C \parallel t$$

۲. روش توصیه شده در علم رمزنگاری نوین: روش Encrypt-then-MAC (EtM) عموماً توصیه می‌شود. این روش امنیت در برابر حملات متن رمز شده انتخابی (CCA) را به طور کامل تضمین می‌کند. گیرنده می‌تواند پیش از انجام عملیات رمزگشایی (که از نظر محاسباتی سنگین است و می‌تواند آسیب‌پذیری‌های امنیتی ایجاد کند)، اصالت و یکپارچگی بسته را بررسی کرده و در صورت دستکاری، آن را فوراً رد کند.

۳. سناریوی حمله (حمله Padding Oracle Attack): در روش MAC-then-Encrypt، اگر مهاجم متن رمز شده را دستکاری کند، سیستم ابتدا مجبور است آن را رمزگشایی کند تا به تگ MAC دسترسی پیدا کند و صحت آن را بسنجد. در حین رمزگشایی، اگر Padding پیام نادرست باشد، سیستم ممکن است خطای متفاوتی نسبت به زمان نادرست بودن MAC صادر کند (یا رفتار زمانی متفاوتی نشان دهد). مهاجم با بهره‌برداری از این تفاوت رفتار (Oracle)، می‌تواند بابت به بایت متن اصلی را بدون داشتن کلید کشف کند. در حالی که در روش EtM، پیام دستکاری شده قبل از هرگونه رمزگشایی به دلیل نامعتبر بودن MAC رد می‌شود و این حمله عملاً غیرممکن می‌گردد.

محدوده اعمال MAC روی بسته‌های شبکه

۱. انتخاب با امنیت بیشتر: اعمال MAC روی ترکیب هدر و بارمفید (h, p) امنیت به مراتب بیشتری را فراهم می‌کند؛ زیرا یکپارچگی کل بسته (هم داده و هم اطلاعات کنترلی) حفظ می‌شود.

۲. توصیف حمله در صورت عدم لحاظ هدر (h): حمله تغییر هدر (Header Modification / Spoofing): اگر MAC فقط روی p اعمال شود، مهاجم در طول مسیر می‌تواند فیلدهای مهم هدر (مانند آدرس آی‌پی مبدأ/مقصد، شماره پورت‌ها یا فیلدهای اولویت) را دستکاری کند. از آنجا که بارمفید تغییر نکرده، تگ MAC همچنان معتبر باقی می‌ماند و گیرنده متوجه دستکاری هدر نخواهد شد. این امر می‌تواند منجر به هدایت اشتباه ترافیک، دور زدن فایروال‌ها یا حملات منع سرویس (DoS) شود.

۳. دلیل رها کردن برخی فیلدهای هدر از محاسبه MAC: دلیل اصلی وجود فیلدهای پویا یا متغیر در طول مسیر (Mutable Fields) است. در پروتکل‌های شبکه، برخی از فیلدهای هدر (مانند TTL یا Time to Live در IP و چک‌سام‌های لایه ۳) در هر گام (Hop) توسط روترهای میانی تغییر می‌کنند. اگر این فیلدها وارد محاسبه MAC شوند، به محض تغییر در اولین روتر میانی، تگ MAC در مقصد نامعتبر شده و بسته دور انداخته می‌شود. بنابراین، طراحان مجبورند فیلدهای متغیر را از محاسبات یکپارچگی سرتاسری (End-to-End) خارج کنند.

پاسخ مسئله‌ی ۶.

استخراج فرمول‌های بازیابی

(آ) فرمول صریح برای بازیابی k : با توجه به اینکه دو امضا با نانس یکسان k و مقدار یکسان r تولید شده‌اند، معادله‌های امضا به صورت زیر هستند:

$$s_1 \equiv k^{-1}(H(M_1) + xr) \pmod{q}$$

$$s_2 \equiv k^{-1}(H(M_2) + xr) \pmod{q}$$

با تفاضل دو معادله خواهیم داشت:

$$s_1 - s_2 \equiv k^{-1}(H(M_1) - H(M_2)) \pmod{q}$$

در نتیجه، فرمول صریح برای بازیابی k برابر است با:

$$k \equiv (s_1 - s_2)^{-1}(H(M_1) - H(M_2)) \pmod{q}$$

(ب) فرمول صریح برای بازیابی کلید خصوصی x : با استفاده از معادله امضای اول:

$$s_1 \equiv k^{-1}(H(M_1) + xr) \pmod{q} \implies k \cdot s_1 \equiv H(M_1) + xr \pmod{q}$$

$$xr \equiv k \cdot s_1 - H(M_1) \pmod{q}$$

با ضرب دو طرف در معکوس ضربی r مرتبه q داریم:

$$x \equiv r^{-1}(k \cdot s_1 - H(M_1)) \pmod{q}$$

محاسبه کلید خصوصی

پارامترهای داده شده:

$$p = 23, \quad q = 11, \quad g = 2, \quad y = 18$$

$$H(M_1) = 4, \quad (r, s_1) = (8, 10)$$

$$H(M_2) = 5, \quad (r, s_2) = (8, 3)$$

(آ) محاسبه مقدار k : ابتدا تفاضل‌ها را در پیمانه $q = 11$ محاسبه می‌کنیم:

$$s_1 - s_2 = 10 - 3 = 7 \pmod{11}$$

$$H(M_1) - H(M_2) = 4 - 5 = -1 \equiv 10 \pmod{11}$$

معکوس ضربی ۷ در پیمانه ۱۱ برابر است با ۸ (زیرا $7 \times 8 = 56 \equiv 1 \pmod{11}$). بنابراین:

$$k \equiv 7^{-1} \times 10 \equiv 8 \times 10 = 80 \equiv 3 \pmod{11}$$

پس مقدار نانس برابر است با: $k = 3$

ب) محاسبه کلید خصوصی آلیس x : ابتدا معکوس ضربی $r = 8$ را در پیمانه ۱۱ پیدا می‌کنیم که برابر ۷ است (زیرا $(8 \times 7 = 56 \equiv 1 \pmod{11})$). حال مقدار داخل پرانتز را محاسبه می‌کنیم:

$$k \cdot s_1 - H(M_1) = 3 \times 10 - 4 = 26 \equiv 4 \pmod{11}$$

بنابراین کلید خصوصی x برابر است با:

$$x \equiv 7 \times 4 = 28 \equiv 6 \pmod{11}$$

پس کلید خصوصی برابر است با: $x = 6$

ج) تایید صحت محاسبات با تولید کلید عمومی: رابطه کلید عمومی در DSS به صورت $y = g^x \pmod{p}$ است:

$$y = 2^6 \pmod{23} = 64 \pmod{23}$$

از آنجا که $64 = 2 \times 23 + 18$ است، داریم:

$$y = 18$$

این مقدار دقیقاً با کلید عمومی داده شده در صورت سوال ($y = 18$) مطابقت دارد، پس محاسبات درست است.

مکانیزم استاندارد جلوگیری از استفاده مجدد Nonce

آ) شناسایی مکانیزم استاندارد: مکانیزم استاندارد صنعتی برای تولید دترمینیستیک (یکتا و معین) نانس در امضاهای DSA و ECDSA در سند RFC 6979 تعریف شده است.

ب) نحوه تولید نانس در این مکانیزم: در این روش، مقدار k به صورت کاملاً دترمینیستیک و بر اساس ترکیب کلید خصوصی (x) و چکیده پیام ($H(M)$) با استفاده از توابع هش رمزنگاری مانند HMAC-SHA256 تولید می‌شود. به این ترتیب، برای یک پیام و کلید خصوصی مشخص، همواره یک k یکتا و کاملاً تصادفی نما تولید خواهد شد.

ج) دلیل از بین رفتن خطرات مرتبط با تولیدکننده‌های اعداد تصادفی ضعیف: از آنجا که فرآیند تولید k دیگر به هیچ منبع سخت‌افزاری یا نرم‌افزاری تولید اعداد تصادفی سیستم در لحظه امضا متکی نیست، خطرات ناشی از آنتروپی ضعیف، نقص‌های فنی یا حملات بهره‌برداری از تولیدکننده تصادفی (RNG) به طور کامل حذف می‌شود. اگر یک پیام دو بار امضا شود، نانس یکسانی تولید شده و امضای کاملاً مشابهی ایجاد می‌کند که هیچ اطلاعاتی در مورد کلید خصوصی فاش نخواهد کرد.

پاسخ مسئله‌ی ۷.

تحلیل دامنه نفوذ

آ) خیر، مهاجم نمی‌تواند یک گواهی معتبر برای یک سرور جعلی EHR جعل کند. سرورهای EHR زیر نظر -Infras tructure Intermediate CA هستند، در حالی که کلید خصوصی فاش‌شده مربوط به User & Device Intermediate CA است. از آنجایی که این دو مرجع صدور مجزا هستند، کلید فاش‌شده توانایی امضا و صدور گواهی برای زیرساخت شبکه و سرورهای EHR را ندارد.

مهاجم تنها می‌تواند گواهی‌های جعلی در دامنه عملکرد User & Device Intermediate CA صادر کند که شامل موارد زیر است:

- گواهی‌های هوشمند شخصی کارکنان (پزشکان و پرستاران) برای ورود به سیستم‌ها یا امضای دیجیتال ایمیل‌ها (S/MIME).
- گواهی‌های مربوط به دستگاه‌های اینترنت اشیاء پزشکی (IoT).

ب) ارزیابی ویژگی‌های امنیتی در حوزه عملکرد کاربران و دستگاه‌های IoT:

- صحت (Integrity): مهاجم با در دست داشتن کلید خصوصی این CA می‌تواند گواهی‌های جعلی S/MIME صادر کند و ایمیل‌های حاوی اطلاعات بیماران را به نام پزشکان یا کارکنان مجاز امضا کند. در نتیجه، گیرنده پیام گمان می‌کند که محتوا دست‌نخورده و معتبر است، در حالی که محتوای ایمیل توسط مهاجم تغییر یافته یا جعل شده است.

- محرمانگی (Confidentiality): مهاجم می‌تواند با صدور گواهی‌های جعلی برای دستگاه‌های IoT پزشکی یا سیستم‌های کاربران، خود را به عنوان یک گیرنده مجاز جا بزند. به عنوان مثال، اگر کاربران ایمیل‌های رمزگذاری‌شده با S/MIME ارسال کنند، مهاجم با گواهی جعلی مرتبط با آن کلید عمومی، قادر به رمزگشایی و شنود اطلاعات حساس بیماران خواهد بود.

- احراز هویت (Authentication): گواهی‌های احراز هویت روی کارت‌های هوشمند کارکنان و گواهی‌های تعبیه‌شده در سخت‌افزار دستگاه‌های IoT برای احراز هویت متقابل با سرورها بر بستر TLS استفاده می‌شوند. مهاجم با جعل این گواهی‌ها می‌تواند به عنوان یک پزشک، پرستار یا یک دستگاه IoT معتبر به سیستم‌های بیمارستان متصل شده و فرآیند احراز هویت را به طور کامل دور بزند.

- عدم انکار (Non-repudiation): از آنجایی که امضای دیجیتال ایمیل‌ها بر پایه این گواهی‌ها بنا شده است، اگر مهاجم ایمیلی مخرب یا دستور درمانی اشتباهی را با یک گواهی جعلی امضا و ارسال کند، سیستم آن را به عنوان امضای معتبر پزشک می‌شناسد. این امر باعث می‌شود که پزشک واقعی نتواند ارسال آن پیام را انکار کند و مسئولیت اقدامات مهاجم به اشتباه متوجه کارکنان شود.

ج) خیر، نیازی به باطل کردن گواهی Root CA یا گواهی‌های صادر شده توسط Infrastructure Intermediate CA وجود ندارد. در ساختار سلسله‌مراتب مراجع صدور (CA Hierarchy)، شاخه‌ها مستقل از یکدیگر عمل می‌کنند. از آنجا که نفوذ تنها به سطح کلید خصوصی User & Device Intermediate CA محدود بوده و به کلیدهای بالا دست (Root CA) یا شاخه‌های موازی سرایت نکرده است، گواهی‌های بخش زیرساخت و سرورهای شبکه همچنان امن و معتبر باقی می‌مانند. تنها باید گواهی خود User & Device Intermediate CA توسط Root CA باطل شود.

ابطال گواهی‌های جعل‌شده

آ) چالش‌ها و محدودیت‌های عملیاتی استفاده از لیست ابطال گواهی (CRL):

- حجم فایل: با توجه به تعداد بسیار زیاد دستگاه‌های IoT و کلاینت‌های کاربری، باطل کردن انبوه گواهی‌ها باعث می‌شود حجم فایل CRL به شدت افزایش یابد.
- پهنای باند شبکه: دانلود دوره‌ای یک فایل CRL حجم متوسط هزاران دستگاه IoT و سیستم کاربری به صورت همزمان، پهنای باند شبکه بیمارستان را اشباع کرده و کارایی ارتباطات حیاتی را کاهش می‌دهد.
- تاخیر (Latency): دستگاه‌ها باید پیش از هر ارتباط، کل فایل CRL را دانلود و در میان لیست طولانی آن جستجو کنند که این امر تاخیر (Latency) فرآیند احراز هویت را به شدت افزایش می‌دهد. همچنین فرآیند به‌روزرسانی CRL زمان‌بر است و در پنجره زمانی انتشار آن، گواهی‌های جعل شده ممکن است همچنان معتبر شناخته شوند.
- ب) پروتکل OSCP به جای ارسال کل لیست ابطال، به کلاینت‌ها اجازه می‌دهد وضعیت یک گواهی خاص را به صورت لحظه‌ای و در قالب یک پاسخ کوچک از سرور استعلام کنند. این کار چالش حجم فایل و پهنای باند را در مقایسه با CRL پوشش می‌دهد.
نقاط ضعف احتمالی OSCP:
- نقطه شکست واحد (SPOF) و بار پردازشی سنگین: مواجهه با حجم عظیم درخواست‌های همزمان از سوی تجهیزات پزشکی، سرور OSCP Responder را زیر بار شدید می‌برد و ممکن است منجر به حملات ناخواسته محرومیت از سرویس (DoS) شود.
- افزایش تاخیر خط ارتباطی: برای هر اتصال TLS، دستگاه باید یک استعلام آنلاین اضافه به سرور OSCP بفرستد و منتظر پاسخ بماند که در تجهیزات پزشکی حساس زمان‌بر است.
- حریم خصوصی: سرور OSCP متوجه می‌شود که کدام دستگاه در چه زمانی به چه سرویسی متصل شده است.
- راهکار بهبود: برای رفع این نقاط ضعف می‌توان از OSCP Stapling استفاده کرد تا خود سرورها وضعیت ابطال گواهی‌شان را ضمیمه کنند و بار از روی سرور استعلام برداشته شود.

بهبودهای پیشنهادی معماری PKI

۱. استفاده از مکانیزم OSCP Stapling: برای کاهش بار سرورهای پاسخ‌دهنده و از بین بردن تاخیر استعلام آنلاین دستگاه‌های IoT، وضعیت اعتبار گواهی به همراه پاسخ دست‌تکانی TLS توسط خود سرور به کلاینت تحویل داده شود.
۲. جداسازی دامنه‌های امنیتی به مراجع صدور میانی بیشتر (Compartmentalization): دستگاه‌های IoT با ریسک بالا را از بخش کاربران عادی تفکیک کرده و برای هر کدام یک Sub-CA مجزا در نظر گرفته شود تا در صورت لو رفتن یک کلید، دامنه نفوذ بسیار کوچک‌تر باشد.
۳. به‌کارگیری ماژول‌های سخت‌افزاری امنیت (HSM): کلیدهای خصوصی تمامی مراجع صدور میانی (Intermediary CAs) باید به جای ذخیره‌سازی روی سیستم عامل سرور، درون سخت‌افزارهای اختصاصی ضد دستکاری (HSM) نگهداری شوند تا امکان استخراج و فاش شدن کلیدها توسط باج‌افزارها به صفر برسد.